SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO2 - ASSESSMENT TOOL - 2

~~Reverse Engineering Task~~

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 2**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO2 Statement	***Requirements Identification, Positive & Negative Testing, Boundary Value Analysis, Equivalence Partitioning ,Defect Identification, Decision Table Testing ,State Transition Testing.***		
Student Name	**A.V.Rama krishna reddy**	**Reg. No.**	**192311123**
Section/Batch	**Slot A**	**Date**	**02-11-2026**

Code of Conduct:

I **A. V.Rama krishna reddy**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Rama krishna

2) A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.

Answer

Introduction

- Reverse engineering in software testing is the process of understanding an existing software system by examining its source code, execution traces, and observed behaviour without having original test documentation.
- In this scenario, the software module's code snippets and execution traces are analyzed to identify requirements, test cases, decision points, execution paths, business rules, and possible defects.
- Objective: To reconstruct a complete software testing strategy using white-box and black-box testing techniques and ensure maximum test coverage.

Identify Functional and Non-Functional Requirements

Requirements are identified by observing the software's behaviour and understanding its operations.

Functional requirements

1. The system should allow users to log in using valid credentials.
2. The system should verify the account balance before transferring money.
3. The system should accept valid beneficiary account details.
4. The system should allow users to enter the transfer amount.
5. The system should process transactions when sufficient balance is available.
6. The system should display success or failure messages.
7. The system should update the account balance after successful transactions.
8. The system should maintain transaction history.

Non-functional requirements

1. Performance: Transactions should be processed within a reasonable time.
2. Security: User credentials and transaction details must be protected.
3. Reliability: Transactions should not result in incorrect balance updates.
4. Availability: Banking services should be available whenever required.
5. Usability: The interface should be simple and user-friendly.
6. Scalability: The system should support multiple users and transactions.

Derive Positive and Negative Test Cases

- Positive test cases verify whether the system works correctly with valid inputs.
- Negative test cases verify how the system handles invalid inputs and unexpected situations.

Test ID	Test scenario	Input	Expected result
TC01	Valid login	Correct username and password	Login successful
TC02	Invalid login	Incorrect password	Error message

TC03	Valid transfer	Amount ₹1,000, balance ₹5,000	Transfer successful
TC04	Insufficient balance	Amount ₹6,000, balance ₹5,000	Transfer rejected
TC05	Invalid beneficiary	Incorrect account number	Error message
TC06	Zero amount	Amount ₹0	Transfer rejected
TC07	Negative amount	Amount -₹500	Transfer rejected
TC08	Valid transaction	Correct details	Balance updated
TC09	Network failure	Connection interrupted	Transaction handled safely
TC10	Empty input	No transaction details	Validation message

Result: Positive and negative test cases help identify incorrect behaviour and improve software reliability.

Identify Boundary Conditions and Design Test Cases

Boundary Value Analysis (BVA) is used to test values at the minimum and maximum limits of an input. For example, assume the banking system allows transfers from ₹100 to ₹50,000 per transaction.

Test ID	Transfer amount	Expected result
TC01	₹99	Invalid
TC02	₹100	Valid
TC03	₹101	Valid
TC04	₹49,999	Valid
TC05	₹50,000	Valid
TC06	₹50,001	Invalid
TC07	₹0	Invalid
TC08	-₹100	Invalid

Explanation:

- Minimum boundary: ₹100.
- Maximum boundary: ₹50,000.
- Values below the minimum should be rejected.
- Values above the maximum should be rejected.
- Values at the boundaries should be accepted.

Divide Inputs into Valid and Invalid Equivalence Classes

Equivalence Partitioning divides input values into groups that are expected to behave similarly.
For example:

Input field	Valid equivalence class	Invalid equivalence class
Username	Registered username	Unregistered username
Password	Correct password	Incorrect or empty password
Account number	Valid account number	Invalid account number
Transfer amount	₹100–₹50,000	Below ₹100 or above ₹50,000
Balance	Sufficient balance	Insufficient balance
Beneficiary	Active beneficiary	Inactive beneficiary

Explanation:
Instead of testing every possible input, one or more representative values from each class are selected. This reduces testing effort while maintaining effective test coverage.

Identify Potential Defects and Explain Their Impact

By analyzing the source code and observed execution traces, possible defects can be identified.

Defect	Possible impact
Incorrect balance calculation	Wrong account balance
Missing input validation	Invalid transactions accepted
Authentication failure	Unauthorized access
Duplicate transaction processing	Multiple deductions
Missing error handling	Application crash
Incorrect beneficiary verification	Money transferred to the wrong account
Network interruption handling failure	Uncertain transaction status
Missing transaction history	Loss of transaction records

Explanation:
Identifying defects helps developers understand the weaknesses of the existing module and improve security, accuracy, and reliability.

Reconstruct Business Rules and Prepare a Decision Table

Business rules define the conditions under which a transaction can be processed.

Identified business rules:

1. The user must be authenticated.
2. The beneficiary account must be valid.
3. The transfer amount must be within the permitted limit.
4. The account must have sufficient balance.
5. The transaction should be processed only when all conditions are satisfied.

Decision table:

Conditions / Rules	R1	R2	R3	R4
User authenticated?	Yes	No	Yes	Yes
Valid beneficiary?	Yes	-	No	Yes
Sufficient balance?	Yes	-	-	No
Transfer amount valid?	Yes	-	-	Yes
Expected action	Transfer	Reject	Reject	Reject

Explanation:

- R1: All conditions are satisfied, so the transfer is successful.
- R2: User is not authenticated, so access is denied.
- R3: Beneficiary is invalid, so the transfer is rejected.
- R4: Balance is insufficient, so the transfer is rejected.

The decision table ensures that important combinations of conditions are tested.

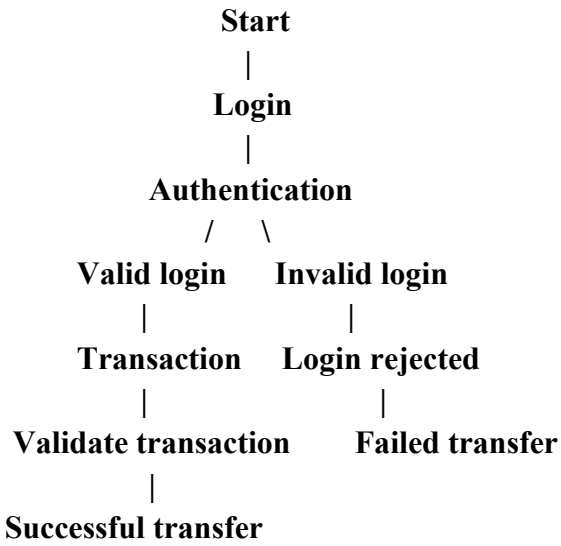
Identify System States, Events, and Valid or Invalid Transitions

State Transition Testing identifies how a system moves from one state to another based on user actions and events.

System states

1. Initial State – User opens the banking application.
2. Login State – User enters login credentials.
3. Authentication State – System verifies credentials.
4. Transaction State – User enters transfer details.
5. Validation State – System checks transaction details.
6. Success State – Transaction is completed.
7. Failure State – Transaction is rejected.

B. State transition diagram



C. Valid transitions

- Start → Login.
- Login → Authentication.
- Authentication → Transaction (valid credentials).
- Transaction → Validation.
- Validation → Success (all conditions satisfied).

D. Invalid transitions

- Login → Transaction without authentication.
- Validation → Success with insufficient balance.
- Validation → Success with an invalid beneficiary.
- Failed transaction → Success without correcting the invalid details.

Result: State transition testing ensures that the system follows the correct sequence of operations and prevents unauthorized or invalid transactions.

White Box Testing – Control Flow Graph and Path Testing

Since the scenario specifically mentions execution traces and code snippets, white-box testing is also necessary.

A. Sample program logic

START
|
Enter login details
|
Is login valid?
| No
v
Display error → END
|
Yes
v
Enter beneficiary and amount
|
Is beneficiary valid?
| No

v
Display error → END
|
Yes
v
Is amount within limit?
| No
v
Display error → END
|
Yes
v
Is balance sufficient?
| No
v
Display insufficient balance → END
|
Yes
v
Process transaction
|
Update balance
|
Display success
|
END

B. Identify decision points

The module contains four major decision points:

1. Login validation.
2. Beneficiary validation.
3. Transfer amount validation.
4. Account balance validation.

Each decision has two outcomes: True and False.

C. Independent execution paths

Path	Execution condition	Expected result
P1	Invalid login	Login rejected
P2	Valid login, invalid beneficiary	Transfer rejected
P3	Valid beneficiary, invalid amount	Transfer rejected
P4	Valid amount, insufficient balance	Transfer rejected
P5	All conditions valid	Transfer successful

These paths represent the independent execution scenarios that should be tested.

D. Coverage criteria

Coverage type	Meaning
Statement coverage	Execute every statement at least once

Branch coverage	Execute True and False outcomes of every decision
Condition coverage	Test each individual condition as True and False
Path coverage	Execute all possible independent execution paths

Test adequacy evaluation:

- If only successful transactions are tested, branch coverage is incomplete.
- If error conditions are not tested, negative behaviour remains unverified.
- If all statements execute but decision outcomes are missing, statement coverage alone is insufficient.
- Independent paths must be tested to improve path coverage.

E. Cyclomatic complexity

Cyclomatic complexity measures the number of independent paths in a program.

Formula:

Where:

- E = Number of edges.
- N = Number of nodes.
- P = Number of connected components (normally 1).

For four independent binary decision points:

Therefore, at least **5 independent paths** should be tested for this sample logic.

Reconstructed Complete Testing Strategy

The following testing activities ensure systematic test coverage.

Testing technique	Testing activity	Objective
White box testing	Control flow analysis	Identify execution paths
Statement coverage	Execute all statements	Verify code execution
Branch coverage	Test True and False branches	Verify decisions
Path testing	Execute independent paths	Verify program logic
Boundary value analysis	Test minimum and maximum amounts	Identify boundary defects
Equivalence partitioning	Divide valid and invalid inputs	Reduce test cases
Decision table testing	Test condition combinations	Verify business rules
State transition testing	Test valid and invalid state changes	Verify system behaviour
Negative testing	Enter invalid inputs	Check error handling

Conclusion

- Reverse engineering helps reconstruct the missing software testing documentation by analyzing source

code, execution traces, and observed system behaviour.

- In this assessment, functional and non-functional requirements, positive and negative test cases, boundary values, equivalence classes, potential defects, business rules, decision tables, and state transitions were identified.
- White-box testing was used to construct the control flow graph, identify decision points, calculate cyclomatic complexity, and determine independent execution paths.
- **Final outcome:** The reconstructed testing strategy improves test coverage, identifies hidden defects, validates business logic, and increases the overall quality, reliability, and security of the software module.

RUBRICS FOR CALCULATION

Assessment Criteria and Performance Levels

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality.	Shows good understanding with minor gaps.	Partial understanding; misses key aspects.	Lacks understanding of the system/product.
Decomposition	25%	Effectively breaks down the system into components with detailed analysis.	Decomposes the system with minor gaps in analysis.	Limited or incomplete decomposition.	Unable to analyze or break down the system.
Identification of Components	20%	Accurately identifies all components and explains their functions clearly.	Identifies most components with minor errors.	Identifies few components; explanations are weak.	Unable to identify components or functions.
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic.	Reconstruction is mostly correct with minor issues.	Partial reconstruction; lacks clarity.	Unable to reconstruct or model the system.
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation.	Documentation is adequate with minor issues.	Poorly organized or incomplete documentation.	No proper documentation or presentation.
Total	100%				